

Frontend Project

Build and Deploy a Full React Application

■ LEARNING OUTCOME

By the end of this module you will design, build, and deploy a complete React application with authentication, API integration, routing, state management, testing, and CI/CD deployment.

■ Skills You Will Gain

- Design component hierarchy from a UI specification
- Set up a production-ready React project
- Implement authentication with protected routes
- Connect to a real API with React Query
- Add testing coverage
- Deploy to Vercel with GitHub Actions

■ One polished deployed React app gets you 5x more interviews than ten unfinished tutorials. Build it properly, write a great README, and it works for you 24/7 as a live recruiter.

PROJECT

Job Board React App

Project Specification

A job board where employers post jobs and candidates browse and apply.

Public: / (landing), /jobs (list+filter), /jobs/:id (detail)
Auth: /login, /register
Protected: /dashboard (employer: manage jobs), /profile (candidate)

Features:
Browse jobs with search, filter by category/location/type
Register/login with JWT auth
Employer: post, edit, delete jobs
Candidate: save jobs, apply
Pagination or infinite scroll

Stack: React + TypeScript + Vite + React Query + React Router + Tailwind + shadcn/ui

SECTION 1

Project Structure

```
src/ main.tsx -- entry point App.tsx -- routes api/ client.ts -- axios instance with interceptors
jobs.ts -- job CRUD functions auth.ts -- auth functions components/ ui/ -- shadcn/ui components
layout/ -- Header, Footer, Layout jobs/ -- JobCard, JobList, JobFilters auth/ -- LoginForm,
RegisterForm pages/ HomePage.tsx JobsPage.tsx JobDetailPage.tsx LoginPage.tsx DashboardPage.tsx
hooks/ useAuth.ts useJobs.ts useDebounce.ts useLocalStorage.ts context/ AuthContext.tsx types/
job.ts user.ts api.ts utils/ formatDate.ts validators.ts e2e/ auth.spec.ts jobs.spec.ts .env.local
vite.config.ts package.json README.md
```

SECTION 2

Key Implementation

```
// types/job.ts export interface Job { id: string; title: string; company: string; location:
string; type: 'full-time' | 'part-time' | 'contract' | 'remote'; category: string; salary: {
min:number; max:number; currency:string }; description: string; postedAt: string; } //
hooks/useJobs.ts export function useJobs(filters: JobFilters) { return useQuery({ queryKey:
['jobs', filters], queryFn: () => api.get('/jobs', { params: filters }).then(r=>r.data), staleTime:
2*60*1000, }); } export function useCreateJob() { const qc = useQueryClient(); return useMutation({
mutationFn: (job) => api.post('/jobs', job).then(r=>r.data), onSuccess: () =>
qc.invalidateQueries({ queryKey:['jobs'] }), }); } // pages/JobsPage.tsx function JobsPage() {
const [filters, setFilters] = useState({ query:'', type:'', category:'' }); const debouncedFilters
= useDebounce(filters, 300); const { data:jobs, isLoading, isError } = useJobs(debouncedFilters);
return ( <div className='container mx-auto px-4 py-8'> <div className='flex flex-col lg:flex-row
gap-8'> <aside className='lg:w-64'> <JobFilters filters={filters} onChange={setFilters} /> </aside>
<main className='flex-1'> <h1 className='text-2xl font-bold mb-6'> {isLoading ? 'Loading...' :
`${jobs?.length||0} Jobs Found` </h1> {isLoading && <JobListSkeleton />} {isError && <ErrorMessage
/>} {jobs && <JobList jobs={jobs} />} </main> </div> </div> ); }
```

SECTION 3

Deployment and Portfolio

1

GitHub

Create public repo. Push code. Add .gitignore for node_modules/, dist/, .env.local

2

Vercel

Connect GitHub at vercel.com. Import project. Auto-detects Vite. Add env vars in dashboard.

3	GitHub Actions Add .github/workflows/ci.yml -- lint + tests on every PR. Block merge on failure.
4	Preview Deploys Every PR gets unique preview URL. Share for review before merging to main.
5	Production Deploy Merge to main auto-deploys. Zero-downtime. Instant rollback to any commit.
6	Monitoring Vercel Analytics for performance. Sentry for error tracking. Lighthouse CI for quality.

Interview Question	Strong Answer
React rendering lifecycle	State change -> reconciliation -> virtual DOM diff -> minimal real DOM update
useMemo vs useCallback	useMemo caches value; useCallback caches function reference for memoized children
Prevent unnecessary re-renders	React.memo, useMemo, useCallback, proper key usage, context splitting
Testing strategy	Unit (utils/hooks), RTL (component interactions), Playwright (critical user flows)
Why React Query over useEffect	Auto caching, background refetch, loading/error, deduplication, optimistic updates
SPA vs SSR	SPA: JS renders; great for dashboards. SSR: server renders; great for SEO and content sites

■ PRACTICE Q & A

Q1. How do you structure a large React application?

A: Feature-based folders: each feature has components, hooks, API calls, tests. Shared UI in components/ui, shared hooks in hooks/, API in api/. Context for global state; React Query for server state.

Q2. What is the difference between server state and client state?

A: Server state: on the server, fetched async (users, products) -- use React Query. Client state: browser only (modal open, form input) -- use useState/useReducer.

Q3. How do you handle loading and error states?

A: Every async needs three states: loading (skeleton), error (message + retry), success (data). React Query provides isLoading, isError, data. Create consistent UI patterns for each.

Q4. What performance optimisations would you apply?

A: React DevTools Profiler first. Then: React.memo for expensive components, useMemo for calculations, lazy routes, virtualisation for long lists, image optimisation.

Q5. What accessibility features must every app have?

A: Semantic HTML, ARIA labels for icon buttons, focus management in modals, keyboard navigation, colour contrast >4.5:1, skip navigation link.

■ Frontend Project Cheat Sheet	
<code>npm create vite@latest app -- --template react-ts</code>	TypeScript React Vite
<code>npx shadcn-ui@latest init</code>	Add shadcn/ui

Feature folder structure	components/, hooks/, api/, pages/, types/
useQuery + useMutation	Server state
ProtectedRoute + AuthContext	Auth guard
import.meta.env.VITE_API_URL	Environment variable
lazy() + Suspense + Routes	Code-split routes
React.memo + useMemo	Prevent re-renders
describe/it/expect + render/screen	Tests
npx playwright test	E2E tests
Lighthouse > 90	Performance target
vercel --prod	Deploy to production